PS 2: Parameter-Efficient Fine-Tuning of a Small Language Model with Iterative Improvement

Course: Conversational AI | Total Marks: 10

General Instructions

1. Read and follow the instructions in every task carefully. Each task carries explicit expectations; missing components will lose marks.
2. **Submission format (mandatory):** All project files must be placed in one folder containing:
    - **A PDF report** with all results, explanations, justifications, and inferences.
    - If submitting a notebook: the **.ipynb file with the output of every executed cell visible**, a **converted .py file**, and the **exported PDF of the executed notebook**.
    - The **saved fine-tuned model or adapter** (or a link to it if size-restricted), with its path documented in the report.
3. **Virtual Lab :** The assignment **must be executed strictly in the Virtual Lab environment only**. Submissions executed outside the prescribed Virtual Lab environment will not be accepted. Full-screen screenshots of the work done in the Virtual Lab environment must be attached separately as proof of execution.
4. For every task, provide a brief explanation, justification, and inference. Code and output alone will not be graded favourably — but keep the writing focused; implementation quality carries more weight than lengthy prose.
5. You may use standard open-source libraries for models, training, and evaluation. However, every configuration value you set (adapter rank, learning rate, epochs, etc.) must be justified in 1–2 lines. Copy-pasted configurations with no reasoning will lose marks.
6. All training runs must be reproducible: fix and report random seeds, and record the exact model checkpoints/versions used.
7. Copied content or unexplained AI-generated answers will be penalized. Cite all datasets, models, papers, and documentation used.

Problem Statement

A company wants an on-premise **domain assistant** that runs on modest hardware. Large hosted LLMs answer domain questions well but are costly, external, and raise data-privacy concerns. Small language models (SLMs) are cheap and private but perform poorly out of the box: weak domain knowledge, poor instruction following, and inconsistent formatting.

Your objective is to test the central claim behind SLM fine-tuning: **can a small model, fine-tuned on domain data with parameter-efficient techniques, close the gap to a much larger pretrained LLM on domain tasks?**

You will prepare an instruction dataset, fine-tune an SLM using a low-rank / quantized technique (LoRA, QLoRA, or a comparable parameter-efficient method), **improve it over at least two training iterations**, and compare three systems throughout:

- **(A)** the base SLM (no fine-tuning),

- **(B)** a much larger pretrained LLM used as a reference upper bound (no fine-tuning),
- **(C)** your fine-tuned SLM.

Domain choice (student's freedom): Choose ONE domain, e.g., banking support, agriculture advisory, campus helpdesk, travel assistance, health & wellness FAQs, developer tool support, or any instructor-approved domain.

Model choice (student's freedom):

- *SLM:* any open model roughly in the **0.5B–3B** range that runs in your environment.
- *Reference LLM:* any substantially larger pretrained model (e.g., a 7B+ open model, or a hosted model accessed through an interface/API). It is used **only for inference as a comparison baseline — do not fine-tune it.**

State both choices and why.

Task 1: Instruction Dataset Preparation — 1.5 Marks

Prepare a flat instruction–response dataset for your domain. Foundation work — do it well, but do not over-invest here.

Your work must include:

- A dataset of **at least 300 examples** with fields: **instruction, optional input/context, expected response**. It may be created manually, adapted from a public dataset, or synthetically generated (disclose the generation method if synthetic).
- Basic cleaning — duplicates, empty/truncated responses, formatting inconsistencies — with before/after counts.
- A train/validation/test split with the ratio stated and justified.
- A one-paragraph note on dataset quality risks (e.g., repetitive phrasing, label noise, coverage gaps) — you will revisit this in Task 4.

Expected Output: Dataset statistics (counts, length distributions), 3 sample records, preprocessing code, and the split.

Don't: spend pages on exploratory analysis — statistics, samples, and the quality note are enough.

Task 2: Baseline Benchmarking — Small Model vs. Large Model — 1.5 Marks

Establish the starting gap between the small and large models before any training.

Your work must include:

- A fixed benchmark of **8 prompts** covering: in-domain factual questions, a formatting-sensitive instruction, a reasoning-style question, and one out-of-scope request. **These same prompts must be reused in Tasks 4 and 5** — freeze them now.
- Responses from **both (A) the base SLM and (B) the reference LLM** on all 8 prompts, recorded verbatim.
- A compact limitations table: where does the SLM fail that the LLM succeeds (domain knowledge, instruction following, formatting, hallucination)? This gap is exactly what fine-tuning must close.

Expected Output: The frozen benchmark prompt set, all A and B outputs, and the gap/limitations table.

Task 3: Parameter-Efficient Fine-Tuning (Iteration 1) — 2.5 Marks

Core component. Fine-tune the SLM using a parameter-efficient technique — and show you understand why it is efficient.

Your work must include:

- Fine-tuning with **LoRA, QLoRA, or a comparable low-rank/quantization-based method**. Briefly explain, in your own words, the idea behind your chosen technique (why training a small number of added parameters on a compressed base model works) — one page maximum, no textbook derivations.
- **Efficiency evidence (mandatory):** report (a) trainable parameters vs. total parameters (count and %), and (b) peak memory during training — and use these numbers to explain why full fine-tuning would not fit the same budget.
- Training configuration (learning rate, epochs, batch size, sequence length, adapter settings) with 1–2 line justifications, plus **training and validation loss curves**.
- The **saved model/adapter** and a working inference demonstration using it.

Expected Output: Training implementation, the two efficiency numbers with explanation, loss curves, saved model path, and an inference demo. This model is your **Version 1 (v1)**.

Don't: pad the report with theory — one paragraph of intuition tied to your measured numbers is worth more.

Task 4: Diagnose and Improve (Iteration 2) — 2.5 Marks

Core component. Fine-tuning is an iterative process, not a single run. Diagnose v1 and produce a better v2.

Your work must include:

- **Diagnosis:** run v1 on the frozen 8-prompt benchmark and on a sample of the validation set. Identify **at least two concrete weaknesses** (e.g., hallucinated facts, ignored format instructions, repetitive answers, over-long responses, failure on out-of-scope prompts), with example outputs as evidence.
- **One or two targeted changes** motivated by the diagnosis — choose from: dataset changes (adding examples of the failing type, cleaning noisy records, rebalancing), training changes (epochs, learning rate, adapter capacity, sequence length), or prompt-format changes. **Each change must be explicitly linked to a diagnosed weakness** ("v1 ignores format instructions → added 40 format-focused examples").
- **Retraining** with the changes to produce **Version 2 (v2)**, with its loss curves and saved path.
- **v1 vs. v2 comparison** on the frozen benchmark: did the targeted weaknesses improve? Did anything get worse? Show at least one before/after output pair per diagnosed weakness.

Expected Output: Diagnosis with evidence, the change log (weakness → change → rationale), v2 training results, and the v1-vs-v2 before/after comparison.

Do: it is acceptable — and instructive — if a change does not help; report it honestly and explain. **Don't:** change five things at once; you won't know what caused the difference.

Task 5: Three-Way Evaluation and Verdict — 2 Marks

Answer the central question with evidence: how close did the fine-tuned SLM get to the large LLM?

Your work must include:

- A **side-by-side comparison of (A) base SLM, (B) reference LLM, and (C) your best fine-tuned SLM (v2)** on the frozen 8-prompt benchmark.
- **Scored evaluation** on at least four criteria (e.g., domain correctness, instruction/format adherence, completeness, hallucination) using a disclosed manual rubric (e.g., 0–2 per criterion) or an LLM-as-judge with the judging prompt disclosed. Report per-system totals.
- **At least one automatic metric** on the held-out test set (e.g., a text-overlap or semantic-similarity score against reference responses) for A vs. C — acknowledge its limitations in one paragraph.
- **Honest error analysis:** at least two cases where the fine-tuned SLM still loses to the reference LLM, with a one-paragraph diagnosis each — is the remaining gap a data problem, a capacity problem, or a training problem?
- A **verdict paragraph:** given quality, cost, privacy, and hardware constraints, when would you deploy the fine-tuned SLM instead of the large LLM — and when not?

Expected Output: The three-way comparison table, rubric/judge scores, automatic metric results, error analysis, and the verdict.

Do: report numbers even if unflattering — analysed failure earns marks. **Don't:** fabricate or cherry-pick outputs.

Learning Outcomes

After completing this assignment, students should be able to:

- Prepare and clean instruction datasets for domain adaptation of language models.
- Apply parameter-efficient fine-tuning (low-rank and quantization-based) and quantify its efficiency.
- Diagnose model weaknesses and run targeted, evidence-linked improvement iterations.
- Compare small, large, and fine-tuned models with rubric-based and automatic evaluation.
- Make a reasoned deployment judgment between a fine-tuned SLM and a large pretrained LLM.

Marks Summary

Task	Component	Marks
1	Instruction dataset preparation	1.5
2	Baseline benchmarking: base SLM vs. reference LLM	1.5
3	**Parameter-efficient fine-tuning (iteration 1) (core)**	2.5
4	**Diagnose and improve (iteration 2) (core)**	2.5

5	Three-way evaluation and verdict	2
	Total	**10**